Web Worker for Heavy Computation

Problem: Build a page that runs a CPU-heavy task (count all primes up to N) without freezing the UI. The work runs in a Web Worker with live progress and a Cancel button. A second button runs the same work on the main thread so the difference is visible.
Tech stack: React (Vite, JavaScript) (default)
Time limit: 60 minutes (default)
Often asked at: Google, Microsoft
Why it matters: keeping the main thread free

PART A: Thought process

Steps that work for any machine coding task:

1. **Clarify** – restate the problem, ask questions, write down assumptions.
2. **List interactions** – every user action and what must happen. This is your test checklist.
3. **Sketch components** – who renders what, who owns which state.
4. **Model state and data** – smallest state that can describe every screen; derive the rest.
5. **Spot the hard parts early** – decide how you will solve them before you type.
6. **Build in layers** – a working static version first, then one behavior at a time, always runnable.
7. **Cover states and edge cases** – idle, running, success, error, cancelled, invalid input.
8. **Wrap up** – trade-offs and what you would do with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases + component / state design	7 min
Pure compute function + main-thread version	8 min
Worker file + message protocol	10 min
`useWorker` hook: progress, result, errors	12 min
Cancel, stale replies, cleanup	10 min
Validation, states, heartbeat demo, wrap-up	8 min

Tip: think out loud. Say *why* the work goes off the main thread, why cancel means `terminate()`, why every message carries a job id. Interviewers grade reasoning, not keystrokes. If you run short on time, say what the next layer would be.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- User enters N and starts a heavy computation (count primes ≤ N, report count, largest prime, time taken).

- The computation runs in a Web Worker; the UI stays responsive (typing, animation, clicks).
- Live progress (0–100%) while it runs.
- User can cancel a running job.
- A "run on main thread" button runs the same code on the UI thread to show the freeze.
- Invalid input is rejected before any work starts.

Non-functional requirements

Area	What it means here
Performance	Main thread never blocked by the compute. Progress messages throttled (≤ ~100 per job) so `postMessage` + re-renders don't flood the UI.
Scalability	One long-lived worker reused across jobs; pattern extends to a pool for many parallel jobs.
Error handling	Errors thrown inside the worker come back as an `error` message; a crashed worker (`onerror`) also lands in the error state.
Accessibility / usability	Labelled input, `aria-invalid` + error text, `aria-live` result region, buttons disabled when the action isn't valid.
Reusability	Compute logic is a pure function, usable in the worker, on the main thread and in tests. The hook hides the worker protocol from components.
Extensibility	Message protocol `{ type, id, ... }` lets you add new task types without changing the hook shape.

Clarifying questions

1. What is the heavy task? Something given, or can I pick one (primes, Fibonacci, sorting a large array, image filter)?
2. Do we need progress, or just a final result?
3. Should starting a new job while one runs cancel the old one, queue it, or be blocked?
4. Is a main-thread comparison needed, or only the worker version?
5. Any upper limit for N? (Stops someone from running a 10-minute job.)
6. Do I need multiple jobs in parallel (worker pool), or one at a time?

Assumptions we will build with

- Task = count primes ≤ N with **trial division** (intentionally slow so the freeze is obvious). N is an integer, 2 ≤ N ≤ 20,000,000.
- Progress is reported in whole percents, at most ~100 messages per job.
- One job at a time. The Run button is disabled while running; if `run()` is called anyway, the newest job wins and the old one is killed.
- Cancel kills the work instantly (`terminate()` + new worker).
- Vite handles bundling the worker with `new Worker(new URL(...), { type: 'module' })` .

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Input prefilled (5,000,000), both cards show "Not run yet", heartbeat dot moving.
Type an invalid value (`''` , `1` , `abc` , `2.5` , `-3` , `99999999`)	Error text under input, `aria-invalid=true` , both Run buttons disabled.
Click **Run in Web Worker**	Card shows "Running… 0%", progress bar fills, Cancel enabled, heartbeat keeps moving, typing still works.
Worker finishes	Card shows count, largest prime and ms taken; Cancel disabled.

Action / input	Expected behavior / output
Click **Cancel** mid-run	Card shows "Cancelled." immediately; no late result ever overwrites it.
Run again after cancel	Works normally (fresh worker).
Click **Run on main thread**	Card shows "Running…", then page freezes: heartbeat stops, typing doesn't show, until the result appears.
Worker throws / crashes	Card shows "Error: …"; Run works again.
Small N (e.g. 100)	Result almost instantly (25 primes, largest 97).
Component unmounts mid-job	Worker terminated, no setState on an unmounted component.

This table is the implementation and test checklist.

3. Design the architecture

Module	Responsibility
`utils/primes.js`	Pure compute (`countPrimes`) and validation (`validateLimit`). No DOM, no worker APIs.
`workers/prime.worker.js`	Thin adapter: receives `{ id, limit }` , calls `countPrimes` , posts `progress` / `done` / `error` .
`hooks/usePrimeWorker.js`	Owns the worker's lifetime and the job state; exposes `{ status, progress, result, error, run, cancel }` .
`components/ResultCard.jsx`	Renders one job's state (idle / running / success / error / cancelled).
`components/Heartbeat.jsx`	JS-driven animation + frame counter: visible proof of whether the main thread is free.
`App.jsx`	Input + validation, buttons, wires the hook and the main-thread run into two ResultCards.

```
App
├── input + error text          (input string state)
├── buttons: Run in Worker / Cancel / Run on main
├── Heartbeat                   (rAF loop, refs only)
├── ResultCard "Web Worker"   ◄── usePrimeWorker ──► prime.worker.js ──► countPrimes
└── ResultCard "Main thread" ◄── local state ──────────────────────────► countPrimes
```

Start simple: get `countPrimes` working on the main thread first. That gives a working app in minutes. Then move the same function into the worker. The pure function is the seam that makes both paths share code.

4. Design the state / data model

State or field	Purpose
`input` (App)	Raw text of the N field. Kept as a string so the user can type freely.
`main` (App)	`{ status, result }` for the main-thread run.
`status` (hook)	`'idle' | 'running' | 'success' | 'error' | 'cancelled'` – one field, no boolean soup.
`progress` (hook)	0–100 for the current job.

State or field	Purpose
`result` (hook)	`{ count, largest, ms }` or `null`.
`error` (hook)	Error message or `null`.
`workerRef` (ref)	The live `Worker` instance. A ref, because swapping workers must not re-render.
`jobIdRef` (ref)	Id of the job we currently care about. Replies with another id are ignored.
`busyRef` (ref)	Whether the worker is mid-job, read synchronously by `run()`.

```
const IDLE = { status: 'idle', progress: 0, result: null, error: null }
const [state, setState] = useState(IDLE)
const workerRef = useRef(null)
const jobIdRef = useRef(0)
const busyRef = useRef(false)
```

Senior note

- **Derived, not stored:** the validation error (`validateLimit(input)`), the numeric limit (`Number(input)`) and button `disabled` flags are computed every render. Storing them would let them drift out of sync.
- **Ownership:** the hook owns everything about the worker (instance, job id, job state). App only owns the input and the main-thread demo. Components below receive plain props.
- **Refs vs state:** anything that only matters to the message handler (job id, busy flag, worker instance) is a ref, so the `onmessage` closure always reads the latest value without re-creating the worker.

5. Design the data structures and contracts

There is no server API. The "API" is the **message protocol** between the main thread and the worker. Every message carries the job `id`.

Main → worker

```
{ id: 3, limit: 5000000 }
```

Worker → main

```
{ type: 'progress', id: 3, progress: 42 }
{ type: 'done',     id: 3, result: { count: 348513, largest: 4999999, ms: 1255 } }
{ type: 'error',    id: 3, message: 'Something went wrong' }
```

Why an `id`? Messages are asynchronous. After a cancel or a new run, a reply from an old job may still be in the queue. The id lets the hook drop it (see challenge 3).

Pure function contract

```
countPrimes(limit: number, onProgress?: (percent: number) => void)
  => { count: number, largest: number | null }

validateLimit(value: string) => string | null   // error message or null
```

Hook contract

```
const { status, progress, result, error, run, cancel } = usePrimeWorker()
run(5_000_000)   // starts (or replaces) a job
cancel()         // kills the current job
```

Sample check values (used in the self-test): `countPrimes(100)` → `{ count: 25, largest: 97 }`, `countPrimes(1_000_000).count` → `78498`.

6. Trace the data flow

Key interaction: **click "Run in Web Worker" with N = 5,000,000**

```
click Run in Web Worker
  |
  ▼
worker.run(5000000)                        (main thread)
  ├─ jobIdRef = 3, busyRef = true
  ├─ setState({ status: 'running', progress: 0 })   → card shows "Running… 0%"
  └─ workerRef.current.postMessage({ id: 3, limit: 5000000 })
  |                                         structured clone → worker thread
  ▼
self.onmessage                             (worker thread)
  └─ countPrimes(5000000, onProgress)
       └─ every 1% → postMessage({ type: 'progress', id: 3, progress: n })
  |                                         ← main thread is free: typing, rAF, clicks run
  ▼
worker.onmessage (progress, id 3 === jobIdRef) → setState(progress) → bar fills
  |
  ▼
loop ends → postMessage({ type: 'done', id: 3, result: { count, largest, ms } })
  |
  ▼
worker.onmessage (done) → busyRef = false → setState({ status: 'success', result })
  |
  ▼
ResultCard: "348,513 primes, largest 4,999,999, took 1255 ms"
```

7. Important technical challenges

Challenge 1: Proving the main thread is actually free

Problem: a CSS spinner is the usual "loading" UI, but CSS `transform` animations can run on the compositor thread and keep spinning even while JavaScript is completely blocked. The demo would lie: both buttons would look "smooth".

Solution: drive the animation from JavaScript with `requestAnimationFrame`. If the main thread is blocked, rAF callbacks don't run, the dot stops and the frame counter stops. Write to the DOM through refs so the 60 fps loop doesn't re-render React.

```
useEffect(() => {
  let frames = 0
  let rafId
  const tick = (t) => {
```

```
    frames++
    dotRef.current.style.transform = `translateX(${(Math.sin(t / 300) + 1) * 120}px)`
    countRef.current.textContent = frames
    rafId = requestAnimationFrame(tick)
  }
  rafId = requestAnimationFrame(tick)
  return () => cancelAnimationFrame(rafId)
}, [])
```

Challenge 2: Cancelling work that is in the middle of a loop

Problem: the worker is inside a `for` loop that runs for seconds. A `postMessage({ type: 'cancel' })` will sit in the worker's queue and only be read **after** the loop ends, so it can't stop anything.

Solution: `worker.terminate()` kills the thread immediately. Then spawn a fresh worker so the next run works. (Alternative if the job must clean up or keep partial results: a `SharedArrayBuffer` flag the loop checks, or chunking the loop with `setTimeout` inside the worker so messages get a turn.)

```
const restart = useCallback(() => {
  jobIdRef.current++          // anything still in flight is now stale
  busyRef.current = false
  workerRef.current.terminate()
  startWorker()
}, [startWorker])

const cancel = useCallback(() => {
  restart()
  setState({ ...IDLE, status: 'cancelled' })
}, [restart])
```

Challenge 3: Stale replies (race conditions)

Problem: user runs job 1, then cancels and runs job 2. A `progress` or `done` message from job 1 might already be posted and waiting in the main thread's queue. Without care, it would overwrite job 2's state with the wrong numbers, or flip a "Cancelled" card back to "success".

Solution: every request gets an incrementing id; every reply echoes it. The handler only accepts replies whose id matches the current `jobIdRef`. Cancel bumps the id, so all older replies are dropped.

```
worker.onmessage = ({ data }) => {
  if (data.id !== jobIdRef.current) return // reply from a cancelled / replaced job
  if (data.type === 'progress') {
    setState((s) => ({ ...s, progress: data.progress }))
    return
  }
  busyRef.current = false
  if (data.type === 'done') setState({ status: 'success', progress: 100, result: data.result, error: null })
  else setState({ ...IDLE, status: 'error', error: data.message })
}
```

Challenge 4: Progress without flooding the main thread

Problem: posting progress for every number means 5 million `postMessage` calls and 5 million `setState`s. Each message is structured-cloned and queued on the main thread, so the "background" work would freeze the UI anyway.

Solution: report only when the integer percent changes – about 100 messages per job, whatever N is.

```
const step = Math.max(1, Math.floor(limit / 100)) // at most ~100 progress messages
for (let n = 2; n <= limit; n++) {
  if (isPrime(n)) { count++; largest = n }
  if (n % step === 0) onProgress(Math.floor((n / limit) * 100))
}
```

8. Implement in the correct order

#	Step	Working result after this step
1	Vite + React scaffold, static layout (input, 3 buttons, 2 cards)	Page renders.
2	`countPrimes` + `validateLimit` as pure functions, quick asserts	Logic verified in Node.
3	Main-thread run with `setTimeout` so "Running…" paints first	Full feature works, but freezes.
4	`Heartbeat` with rAF + refs	The freeze is visible.
5	`prime.worker.js` + `new Worker(new URL(...), { type: 'module' })`	Worker computes; result logged.
6	`usePrimeWorker`: create in effect, terminate in cleanup, `run`, `onmessage` → state	Worker run updates the card, UI stays smooth.
7	Progress messages (throttled) + `<progress>` bar	Live progress.
8	Job ids + stale-reply check	No race between jobs.
9	`cancel` = terminate + recreate	Instant cancel.
10	Error paths: try/catch in worker, `onerror` in hook	Error state shown.
11	Validation UI, disabled buttons, ARIA	Edge cases and a11y covered.

9. Cover all states and edge cases

State / edge case	Handling
Initial / idle	"Not run yet." Cancel disabled.
Running (worker)	"Running… N%", progress bar, Run disabled, Cancel enabled.
Running (main)	"Running…" painted first via `setTimeout(…, 50)`, then the thread blocks.
Success	Count, largest prime, duration.
Cancelled	"Cancelled." Old replies ignored by job id.
Error in compute	Worker catches it, posts `{ type: 'error' }`.
Worker crash / load failure	`worker.onerror` → error state, `preventDefault()` keeps it out of the console as uncaught.

State / edge case	Handling
Empty input / non-number / decimal / < 2	`validateLimit` message, buttons disabled.
N too large	"Max is 20,000,000" – protects users from multi-minute jobs.
N = 2	1 prime, largest 2 (smallest valid input).
Run called while busy	Old worker terminated, newest job wins.
Unmount mid-job	Effect cleanup terminates the worker; no late setState.
React StrictMode double effect	Effect creates the worker and cleanup terminates it, so the second mount gets a clean worker.
Browser has no Worker support	Not handled (all evergreen browsers support it); fallback would be the main-thread path.

10. Final structure

```
_94_web_worker_heavy_computation/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx
    ├── components/
    │   ├── Heartbeat.jsx
    │   └── ResultCard.jsx
    ├── hooks/
    │   └── usePrimeWorker.js
    ├── workers/
    │   └── prime.worker.js
    └── utils/
        └── primes.js
```

11. Full working code

Setup is the standard Vite React template with Tailwind (`@tailwindcss/vite`). No extra dependencies: Web Workers are a browser API, and Vite bundles the worker file from the `new URL(..., import.meta.url)` pattern.

src/utils/primes.js

```
export const MAX_LIMIT = 20_000_000

export function validateLimit(value) {
  const n = Number(value)
  if (value === '' || !Number.isInteger(n) || n < 2) return 'Enter a whole number of 2 or more'
  if (n > MAX_LIMIT) return `Max is ${MAX_LIMIT.toLocaleString()}`
  return null
}
```

```
function isPrime(n) {
  if (n % 2 === 0) return n === 2
  for (let d = 3; d * d <= n; d += 2) if (n % d === 0) return false
  return true
}

// Deliberately slow (trial division) so the work is heavy enough to block the UI.
export function countPrimes(limit, onProgress = () => {}) {
  let count = 0
  let largest = null
  const step = Math.max(1, Math.floor(limit / 100)) // at most ~100 progress messages
  for (let n = 2; n <= limit; n++) {
    if (isPrime(n)) {
      count++
      largest = n
    }
    if (n % step === 0) onProgress(Math.floor((n / limit) * 100))
  }
  return { count, largest }
}
```

src/workers/prime.worker.js

```
import { countPrimes } from '../utils/primes.js'

self.onmessage = ({ data: { id, limit } }) => {
  try {
    const start = performance.now()
    const result = countPrimes(limit, (progress) => self.postMessage({ type: 'progress', id, progress }))
    const ms = Math.round(performance.now() - start)
    self.postMessage({ type: 'done', id, result: { ...result, ms } })
  } catch (err) {
    self.postMessage({ type: 'error', id, message: err.message })
  }
}
```

src/hooks/usePrimeWorker.js

```
import { useCallback, useEffect, useRef, useState } from 'react'

const IDLE = { status: 'idle', progress: 0, result: null, error: null }

export default function usePrimeWorker() {
  const workerRef = useRef(null)
  const jobIdRef = useRef(0)
  const busyRef = useRef(false)
  const [state, setState] = useState(IDLE)

  const startWorker = useCallback(() => {
    const worker = new Worker(new URL('../workers/prime.worker.js', import.meta.url), { type: 'module' })
    worker.onmessage = ({ data }) => {
      if (data.id !== jobIdRef.current) return // reply from a cancelled / replaced job
      if (data.type === 'progress') {
```

```
        setState((s) => ({ ...s, progress: data.progress }))
        return
      }
      busyRef.current = false
      if (data.type === 'done') setState({ status: 'success', progress: 100, result: data.result, error: null })
      else setState({ ...IDLE, status: 'error', error: data.message })
    }
    worker.onerror = (e) => {
      e.preventDefault()
      busyRef.current = false
      setState({ ...IDLE, status: 'error', error: e.message || 'Worker crashed' })
    }
    workerRef.current = worker
  }, [])

  useEffect(() => {
    startWorker()
    return () => workerRef.current.terminate()
  }, [startWorker])

  // A worker can't be interrupted mid-loop, so cancel = kill it and spawn a fresh one.
  const restart = useCallback(() => {
    jobIdRef.current++
    busyRef.current = false
    workerRef.current.terminate()
    startWorker()
  }, [startWorker])

  const run = useCallback(
    (limit) => {
      if (busyRef.current) restart() // newest request wins
      const id = ++jobIdRef.current
      busyRef.current = true
      setState({ ...IDLE, status: 'running' })
      workerRef.current.postMessage({ id, limit })
    },
    [restart],
  )

  const cancel = useCallback(() => {
    restart()
    setState({ ...IDLE, status: 'cancelled' })
  }, [restart])

  return { ...state, run, cancel }
}
```

src/components/Heartbeat.jsx

```
import { useEffect, useRef } from 'react'

// JS-driven on purpose: CSS animations can run on the compositor and keep moving while the main thread is frozen.
export default function Heartbeat() {
  const dotRef = useRef(null)
```

```
  const countRef = useRef(null)

  useEffect(() => {
    let frames = 0
    let rafId
    const tick = (t) => {
      frames++
      dotRef.current.style.transform = `translateX(${(Math.sin(t / 300) + 1) * 120}px)`
      countRef.current.textContent = frames
      rafId = requestAnimationFrame(tick)
    }
    rafId = requestAnimationFrame(tick)
    return () => cancelAnimationFrame(rafId)
  }, [])

  return (
    <div className="rounded-lg border border-slate-200 bg-white p-4">
      <p className="text-sm text-slate-600">
        Main thread heartbeat — frames painted: <span ref={countRef} data-testid="frames">0</span>
      </p>
      <div className="mt-3 h-4 w-64">
        <div ref={dotRef} className="h-4 w-4 rounded-full bg-emerald-500" />
      </div>
    </div>
  )
}
```

src/components/ResultCard.jsx

```
export default function ResultCard({ title, status, progress, result, error, showProgress }) {
  return (
    <section className="rounded-lg border border-slate-200 bg-white p-4" aria-live="polite" data-testid={title}>
      <h2 className="font-semibold">{title}</h2>
      {status === 'idle' && <p className="text-sm text-slate-500">Not run yet.</p>}
      {status === 'running' && (
        <>
          <p className="text-sm text-amber-700">Running…{showProgress && ` ${progress}%`}</p>
          {showProgress && <progress className="mt-2 w-full" max={100} value={progress} />}
        </>
      )}
      {status === 'cancelled' && <p className="text-sm text-slate-500">Cancelled.</p>}
      {status === 'error' && <p className="text-sm text-red-600">Error: {error}</p>}
      {status === 'success' && (
        <p className="text-sm">
          <b>{result.count.toLocaleString()}</b> primes, largest {result.largest?.toLocaleString() ?? '—'}, took{'
'}
          {result.ms} ms
        </p>
      )}
    </section>
  )
}
```

src/App.jsx

```
import { useState } from 'react'
import Heartbeat from './components/Heartbeat.jsx'
import ResultCard from './components/ResultCard.jsx'
import usePrimeWorker from './hooks/usePrimeWorker.js'
import { countPrimes, validateLimit } from './utils/primes.js'

export default function App() {
  const [input, setInput] = useState('5000000')
  const [main, setMain] = useState({ status: 'idle', result: null })
  const worker = usePrimeWorker()

  const error = validateLimit(input)
  const limit = Number(input)

  const runOnMain = () => {
    setMain({ status: 'running', result: null })
    // Let React paint "Running…" before the loop freezes the thread.
    setTimeout(() => {
      const start = performance.now()
      const result = countPrimes(limit)
      setMain({ status: 'success', result: { ...result, ms: Math.round(performance.now() - start) } })
    }, 50)
  }

  return (
    <main className="mx-auto max-w-2xl space-y-4 p-6">
      <h1 className="text-2xl font-bold">Web Worker: heavy computation</h1>

      <form className="space-y-2" onSubmit={(e) => e.preventDefault()}>
        <label htmlFor="limit" className="block text-sm font-medium">
          Count primes up to
        </label>
        <input
          id="limit"
          inputMode="numeric"
          value={input}
          onChange={(e) => setInput(e.target.value.trim())}
          aria-invalid={!!error}
          aria-describedby="limit-error"
          className="w-full rounded border border-slate-300 px-3 py-2"
        />
        <p id="limit-error" className="min-h-5 text-sm text-red-600">
          {error}
        </p>
        <div className="flex flex-wrap gap-2">
          <button
            type="button"
            disabled={!!error || worker.status === 'running'}
            onClick={() => worker.run(limit)}
            className="rounded bg-emerald-600 px-4 py-2 text-white disabled:opacity-50"
          >
            Run in Web Worker
          </button>
          <button
```

```
            type="button"
            disabled={worker.status !== 'running'}
            onClick={worker.cancel}
            className="rounded border border-slate-300 px-4 py-2 disabled:opacity-50"
          >
            Cancel
          </button>
          <button
            type="button"
            disabled={!!error || main.status === 'running'}
            onClick={runOnMain}
            className="rounded bg-red-600 px-4 py-2 text-white disabled:opacity-50"
          >
            Run on main thread (freezes UI)
          </button>
        </div>
      </form>

      <Heartbeat />

      <label className="block text-sm">
        Type here while it runs:
        <input className="mt-1 w-full rounded border border-slate-300 px-3 py-2" placeholder="Still responsive?" />
      </label>

      <ResultCard title="Web Worker" showProgress {...worker} />
      <ResultCard title="Main thread" {...main} />
    </main>
  )
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/index.css

```
@import "tailwindcss";

body {
  background: #f8fafc;
  color: #0f172a;
}
```

Quick self-check (Node)

```
import assert from 'node:assert'
import { countPrimes, validateLimit } from './src/utils/primes.js'

assert.deepStrictEqual(countPrimes(100), { count: 25, largest: 97 })
assert.deepStrictEqual(countPrimes(2), { count: 1, largest: 2 })
assert.strictEqual(countPrimes(1_000_000).count, 78498)
const seen = []
countPrimes(1000, (p) => seen.push(p))
assert.ok(seen.length <= 101 && seen.at(-1) === 100)
assert.strictEqual(validateLimit('5000000'), null)
for (const v of ['', '1', 'abc', '2.5', '-3', '99999999']) assert.ok(validateLimit(v))
```

Measured in the browser with N = 5,000,000: main-thread run blocked the page for ~1,070 ms in one piece; the worker run took ~1,250 ms but the longest main-thread gap during it was ~19 ms.

12. Wrap-up

Edge cases handled

- Invalid, empty, decimal, negative and too-large input.
- Cancel mid-run, run while busy, stale replies from old jobs.
- Errors inside the worker and a crashed worker.
- Unmount and StrictMode double-mount (no leaked workers).
- Progress throttled so the worker can't flood the UI.

Trade-offs to mention

- **Terminate vs cooperative cancel:** `terminate()` is instant and simple but loses partial results and any cleanup. Cooperative cancel (shared flag or chunked loop) keeps them but adds code and slows the hot loop.
- **One worker reused vs one per job:** reuse avoids ~tens of ms startup per job; per-job is simpler but wasteful. We reuse and only recreate on cancel.
- **Trial division on purpose:** a sieve would be far faster. The slow algorithm is the point of the demo. In a real app, first make the algorithm faster, then move what's still heavy off the main thread.
- **Structured clone cost:** inputs/outputs here are tiny. For big arrays use `Transferable` (`postMessage(buf, [buf])`) to move instead of copy.
- **Worker adds ~1 thread + bundle chunk:** worth it only for work that takes more than ~50 ms (a long task).

With more time

- Generic `useWorker(workerFactory)` hook or Comlink to hide `postMessage` behind async function calls.
- Worker pool (`navigator.hardwareConcurrency`) to split the range across cores and merge counts.
- Cache results per N (`Map`) so repeat runs are instant.
- Tests: Vitest for `primes.js` ; mock `Worker` class to test the hook's stale-id and cancel logic; Playwright to assert the heartbeat keeps moving.
- ETA from progress rate; show elapsed time while running.

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use `async/await` or a Promise instead of a worker?

Promises don't create a new thread. An `async` function still runs its synchronous loop on the main thread, so the page freezes the same way. Async only helps when you are *waiting* (network, timers), not *computing*.

Q2. When is a Web Worker worth it?

When a piece of synchronous work takes longer than ~50 ms (a "long task"): parsing big JSON/CSV, image processing, search indexing, crypto, diffing, heavy formatting. For short work, the messaging and startup overhead isn't worth it.

Q3. What can't a worker do?

No DOM, no `window`, no `document`, no React. It has `fetch`, timers, `IndexedDB`, `WebSocket`, `OffscreenCanvas`, `crypto`. So only pure computation and I/O go to the worker; rendering stays on the main thread.

State design

Q4. Why is the worker instance in a ref and not in state?

Replacing the worker (on cancel) shouldn't cause a render, and the message handler needs the current instance without being re-created. A ref gives a stable, mutable box. Only what the UI shows (status, progress, result, error) is state.

Q5. Why a single `status` string instead of `isLoading`, `isError`, `isCancelled` booleans?

Booleans allow impossible combinations (`isLoading && isError`). One status with a fixed set of values makes each render branch explicit and impossible states unrepresentable.

Q6. Why keep the input as a string?

So the user can type freely (empty field, partial numbers) without the value jumping. We derive the number and the validation error from it on each render.

Challenge: freeze proof

Q7. Why doesn't a CSS spinner prove anything?

Transform/opacity animations can run on the compositor thread, so they keep moving while JS is blocked. A `requestAnimationFrame` loop runs on the main thread, so it stops exactly when the page is frozen.

Q8. How would you measure the freeze, not just see it?

`PerformanceObserver` with `entryTypes: ['longtask']`, or a `setInterval(…, 10)` that records the largest gap between ticks. In our run: ~1,070 ms gap on the main thread vs ~19 ms with the worker.

Challenge: cancel

Q9. Why doesn't sending a `{ type: 'cancel' }` message work?

The worker is single-threaded too. Its `onmessage` only runs after the current task (the long loop) returns, so the cancel message is read after the job is already done.

Q10. What if you need partial results on cancel?

Use cooperative cancel: process in chunks and yield (`await new Promise(r => setTimeout(r))`) between chunks so a cancel message can be read, or check a flag in a `SharedArrayBuffer` via `Atomics.load` (needs cross-origin isolation headers). Then post the partial result before stopping.

Q11. Isn't recreating a worker on every cancel expensive?

It costs a few to tens of ms (thread + script load, usually from cache). Cancel is a rare user action, so that's fine. If cancels were frequent, use cooperative cancel and keep the worker.

Challenge: races and messages

Q12. Why does every message carry an id if we already terminate on cancel?

`terminate()` stops future messages, but replies already posted can still be sitting in the main thread's queue. The id check drops them. It also covers the "run while busy" path.

Q13. How is data passed to the worker, and what's the cost?

`postMessage` uses the structured clone algorithm: it copies the data. Small objects are cheap. For large binary data, transfer ownership instead: `worker.postMessage(buffer, [buffer])` – zero copy, but the sender can no longer use it.

Q14. What if the function you send contains a callback or class instance?

Functions can't be cloned (`DataCloneError`); class instances lose their prototype. Send plain data only. If you want function-call style, use Comlink, which wraps this in proxies.

Implementation steps

Q15. Why `new Worker(new URL('./x.js', import.meta.url), { type: 'module' })` ?

That exact pattern lets Vite (and webpack 5) find the file, bundle it as a separate chunk with its imports, and give it a hashed URL. `type: 'module'` lets the worker use `import` so it can share `primes.js` with the main thread.

Q16. Why is the worker created in `useEffect` and not at module level?

The effect ties the worker's life to the component: created on mount, terminated on unmount. A module-level worker would live forever and be shared across every component instance. In StrictMode the effect runs twice in dev; cleanup terminates the first worker, so nothing leaks.

Q17. Why the `setTimeout(…, 50)` before the main-thread run?

`setMain({ status: 'running' })` only schedules a render. If the loop started immediately, the browser would never paint "Running…" before freezing. Deferring lets React commit and the browser paint first.

Performance and "what if" twists

Q18. What if N is 1 billion?

Trial division would take hours. First change the algorithm: a segmented Sieve of Eratosthenes (memory per segment stays small). Then split the range across a worker pool. Also raise the input limit only after that.

Q19. How would you use all CPU cores?

Create `navigator.hardwareConcurrency - 1` workers, split `[2, N]` into ranges, send each a `{ from, to }`, sum the counts and take the max `largest`. Progress = sum of each worker's processed count / total.

Q20. What if many components need heavy work?

Build a shared worker pool service with a task queue: `pool.run(task) → Promise`, each task with an id. Components use a small hook on top. Limits concurrency and reuses threads.

Q21. What if the user switches tabs during a long job?

Workers keep running in background tabs (timers on the main thread get throttled, workers mostly don't). When the tab returns, the latest result is already in state. If battery matters, pause on `visibilitychange`.

Trade-offs and "why not X?"

Q22. Why not `requestIdleCallback` or chunking on the main thread?

That works: split the loop into ~5 ms chunks and yield between them. It keeps the UI responsive without a worker but makes the job slower and the code more complex, and still uses the main thread's time. A worker runs at full speed on another core.

Q23. Why not a SharedWorker or Service Worker?

A SharedWorker is shared across tabs of the same origin – useful for one shared connection, not for per-page computation. A Service Worker is a network proxy with an event-driven lifetime the browser can kill anytime – wrong tool for long CPU work.

Q24. Why not WebAssembly?

WASM can make the compute faster, but it still blocks whichever thread runs it. It's complementary: run the WASM inside a worker. For this task plain JS is fast enough to explain the concept.

Q25. How would you test this?

Unit-test `primes.js` directly (it's pure). Test the hook with a fake `Worker` class that records `postMessage` and lets the test fire `onmessage` with chosen ids – check that stale ids are ignored and cancel calls `terminate`. End-to-end, run in Playwright and assert the heartbeat frame count keeps rising during the worker run.